CS683: Advanced Computer Architecture

Programming Assignment 2

Team Members

Abhinav V	24B1007
Boominadharshan K	24B1225
Sabarinath S	24B1052
Santhosh BV	24B1290

Contents

1 Task 1 **1**
1.1 Task-1A 1
1.2 Task-1B 1

2 Task 2 **6**
2.1 Task 2a 6
2.2 Task 2b 7

3 Task 3 **12**

1 Task 1

Table 1: Baseline Performance (No Prefetcher)

Trace	IPC	L1D Load MPKI	L1D Misses
Trace 1	0.232345	15.1755	379388
Trace 2	0.251158	18.7336	468340
Trace 3	0.382065	5.03724	125931

1.1 Task-1A

Table 2: IP-Stride Prefetcher Performance

Trace	IPC	Speedup	L1D Load MPKI	Prefetch Accuracy (%)	L1D Coverage (%)
Trace 1	0.233452	1.0048×	15.5962	52.7133	-2.7724
Trace 2	0.251260	1.0004×	19.0268	38.1823	-1.5649
Trace 3	0.382862	1.0021×	5.49224	15.9918	-9.0327

1.2 Task-1B

For each instruction we maintain a class to hold and track the information, it contains:

- A *ip* field to store the masked ip address of the instruction.
- A *last_cl_addr* field to store the last line address
- A *uni_stride* says the stride of ip in case of uniform pattern.
- A *uni_conf* fields say confidence for the uniform pattern else it falls back to non uniform prefetching.
- The *valid* bit to indicate is the tracer valid or stats for some instruction.
- A *lru* field to say how recently used is this ip among other ip's.
- *sig* field to store last k strides into a 16 bits. Those strides are hashed together store here. It stores the signature of last stride.
- *pat_sig* to store many such signatures here.
- *pat_valid* array to say is the signature is valid or not.

- *pat_conf* array to say confidence of each such signature.
- *pat_stride* array to store the next stride if the signature matches.
- *pat_next* says if current signature mismatches then what next signature/pattern to search.

The constants or the parameters that can be tuned are

- $SPP_TRACKER_COUNT$ says size of the tracer array
- $MASK_IP$ is the bits of ip that are used to hash the ip address as a key.
- UNI_CONF_MAX field to say the maxmum confidence achievable incase of uniform prefetching.
- UNI_CONF_THRESH to say the threshold of the uniform prefetching to take.
- $LINES_PER_PAGE$ fixed quantity say number of lines per page.
- SIG_BITS, SIG_MASK says number of bits in the mask of signature used as a key to hash.
- PAT_WAYS number of signatures to store. PAT_MAX as maximum confidence achievable. PAT_THRESH to say threshold of confidence to issue the prefetch.
- $THROTTLE_SAMPLE_SZ$ to indicate when to throttle the degree of prefetch and calculate the latest accuracy.

Pattern find function finds the correct signature which matches the current signature. Returns the index if found a valid match. Pattern victim finds the invalid pattern first if not found finds a pattern where confidence is zero using round robin eviction. If no such found we decrement confidence of all patterns and returns the pattern using pattern next.

Pattern update updates confidence of pattern if signature matches else finds a victim to insert the new signature sets the new signature and stride of the pattern. Update lru is called when we insert a new tracker or when reused it increments lru of all trackers whose lru is below it and sets its lru to 0.

Next signature finds the signature using old signature and new stride, it hashes and encode the signature using both with the signature mask bits. The prefetch operator calls accuracy throttling at very first to increase prefetch degree based on line address and ip using the mask finds the correct traker which matches ip with current ip mask. If such match found we evict the tracker. Based on the lru and validity of the tracker, initializes the all parameters. If match found we update its lru based on reuse and calculate the stride of line address in the page with old line address seen. If the line addresses cross a page we reset the signature then return. If the stride matches with the last uniform stride we prioritize uniform prefetching and update the confidence of uniform prefetch and if the confidence is more than threshold we issue prefetching based on this uniform prefetcher with the throttled current degree.

If the stride of uniform is not matched with the current stride, confidence is decremented. Similar to ip stride prefetcher we issue prefetch to L1D only if mshr is half full else its isssued to lower levels. If threshold of uniform prefetcher is not met we fall back to non uniform prefetcher. Here we first update the pattern based on the current stride and old signature used, it then probes next signature to get the signature to use for this instruction. Using this signature we find the pattern index in the pattern tracker, which inturn gives the next stride that is used to issue prefetch.

Remaining checks like prefetch within page and fill L1D if mshr is less than half filled remains same as ip stride. This prefetching is done to the current degree thus in every iteration we have to update the signature and find its corresponding pattern index to get our stride to prefetch. delta holds that sum of strides from the start of current line address.

Accuracy throttling is done in every 100 prefetch instructions where we find number of useful prefetches and number of prefetches to lower level. It work like memory less for a current window

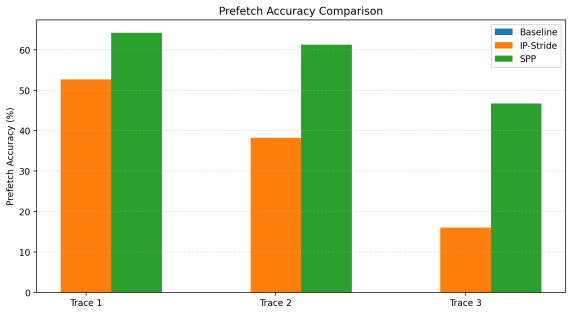


Figure 1: Prefetch Accuracy of different methods

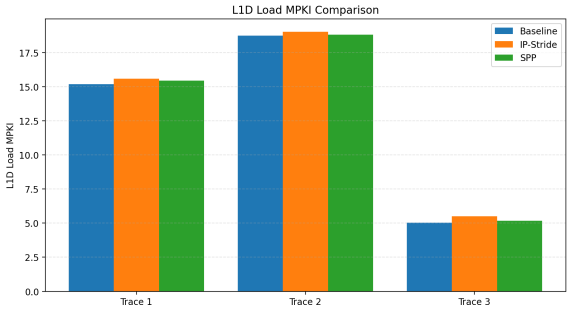


Figure 2: L1D Load MPKI of different methods

size of 100 we find number of useful and total prefetches tocalculate the accuracy of prefetcher in this current window size. Based on ths accuracy we set the degree like if acc=0.8 deg=6, acc=0.45 deg=3 so on. To track the number of useful prefetches, current degree, total prefetches in current window we have global static variables correspondingly. We don't do throttling when the current useful prefetches/total prefetches is less than last useful prefetches/total prefetches in the last window.

Table 3: SPP Prefetcher Performance

Trace	IPC	Speedup	L1D Load MPKI	Prefetch Accuracy (%)	L1D Coverage (%)
Trace 1	0.233369	1.0044×	15.4406	64.1979	-1.7468
Trace 2	0.251399	1.0010×	18.8072	61.2218	-0.3931
Trace 3	0.383352	1.0023×	5.17784	46.7029	-2.7912

A five level thresholding for throttling is done with different prefetch degrees. The value were finalized over different experiments involving varying prefetch degrees and levels of thresholding. For a particular number of thresholding levels as the prefetch degree was increased it was observed that the accuracy and coverage of the prefetcher improved. However when the prefetch degree at lower confidence was kept at 4,5,6 the coverage began to drop and the accuracy of the prefetcher also decreased. This also affected the percentage of the prefetch requests which arrived later. A similar trend was observed when the prefetch degree was increased to 8,9,10 when the accuracy was high. This behaviour can be attributed to the increasing cache pollution at the L1 and L2 levels. However at high accuracy the increasing prefetch degree got higher hits in the L2 compared to lower degree. This improvement also decreased with aggressive prefetching at higher degree as explained above. The accuracy values to choose the thresholding interval was optimized along with the prefetch degree.

Similarly number of threshold levels were varied over different experiments. With the increase

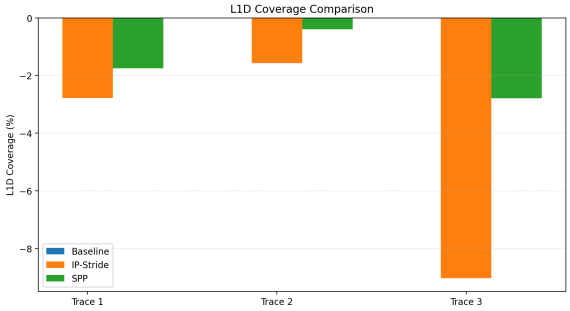


Figure 3: L1D coverage of different methods

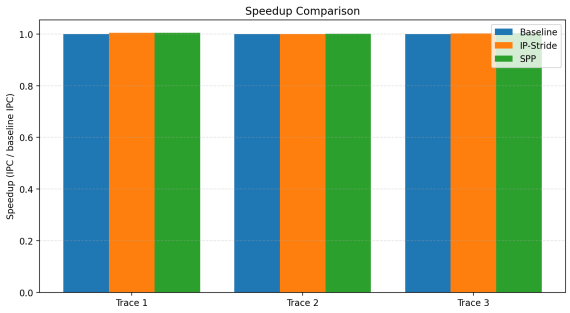


Figure 4: L1D speedup of different methods

Table 4: Memory Overhead for SPP tracker

Structure	**Variable Field**	**Count / Width**	**Subtotal (bits)**
Global Control Registers	Useful Counter Snap (`last_useful_cnt`)	64 bits	64 bits
	Lower Level Counter Snap (`last_lower_cnt`)	64 bits	64 bits
	Active Degree Register (`current_degree`)	3 bits	3 bits
	Accuracy State Register (`latest_accuracy`)	8 bits	8 bits
Total Global Control Overhead			**139 bits (≈ 18 Bytes)**
Base Tracker Entry	Valid Bit (`valid`)	1 bit	1 bit
	Instruction Pointer Tag (`ip`)	10 bits	10 bits
	Last Line Address (`last_cl_addr`)	58 bits	58 bits
	Uniform Stride (`uni_stride`)	7 bits	7 bits
	Uniform Confidence (`uni_conf`)	2 bits	2 bits
	Current Signature (`sig`)	12 bits	12 bits
	Replacement LRU State (`lru`)	6 bits	6 bits
	Pattern Valid Mask (`pat_valid`)	8 bits	8 bits
	Pattern Replacement Pointer (`pat_next`)	3 bits	3 bits
Pattern Table (8 Ways)	Way Signatures (`pat_sig`)	8×12 bits	96 bits
	Way Strides (`pat_stride`)	8×7 bits	56 bits
	Way Confidence Counters (`pat_conf`)	8×2 bits	16 bits
Single SPP_TRACKER Entry Overhead			**273 bits (35 Bytes)**
Tracker Table Overhead (64×273 **bits**)			**17,472 bits (2,184 Bytes)**
Total Hardware Overhead (Global + 64 Trackers)			**17,611 bits (2,202 Bytes ≈ 2.15 KiB)**

Accuracy Threshold (A)	**Prefetch Degree**
$A \geq 0.80$	6
$0.60 \leq A < 0.80$	4
$0.45 \leq A < 0.60$	3
$0.25 \leq A < 0.45$	2
$A < 0.25$	1

Table 5: Accuracy thresholds and corresponding prefetch degrees

in the number of thresholding levels improvement in the prefetcher performance was observed as a higher degree of control in terms of choosing different prefetch degree is obtained. However the gains decreased with more fine thresholding levels and it became difficult to optimize the accuracy values for such increased number of levels.

2 Task 2

Table 6: LRU Baseline Performance

Trace	IPC	L2C Load MPKI	L2 Instruction MPKI	L2 Data MPKI
Trace 1	0.232345	17.69560	10.33770	7.35796
Trace 2	0.251158	27.35680	18.85330	8.50352
Trace 3	0.382065	3.95424	1.12408	2.83016

2.1 Task 2a

Under our custom instruction-aware RRIP implementation, a few key modifications are made. They are summarized below.

Initialization Upon startup, every cache line is initialized with the maximum RRPV value, as none of the lines is initially given a preference or priority for becoming the next victim. Hence, every line is initially equally preferred for eviction.

```
for (int i = 0; i < L2C_SET; i++) {
    for (int j = 0; j < L2C_WAY; j++) {
        l2_rrpv[i][j] = maxRRPV;
    }
}
```

Listing 1: Initialization of RRPV state

Victim Selection Victim selection is handled as follows.

```
while (1) {
    for (int i = 0; i < L2C_WAY; i++)
        if (l2_rrpv[set][i] == maxRRPV &&
            current_set[i].instruction == 0)
            return i;

    for (int i = 0; i < L2C_WAY; i++)
        if (l2_rrpv[set][i] == maxRRPV &&
            current_set[i].instruction == 1)
            return i;

    for (int i = 0; i < L2C_WAY; i++)
        l2_rrpv[set][i]++;
}
```

Listing 2: Finding a victim using instruction-aware RRIP

The victim selection logic follows these steps:

- Lines with the maximum RRPV are searched first. Among them, non-instruction lines are preferred for eviction, thereby preserving instruction lines whenever a suitable non-instruction victim is available.
- Instruction lines are evicted only when no non-instruction line with the maximum RRPV can be found.
- If no line has the maximum RRPV, the RRPV of every way in the set is incremented until a suitable victim is found.

Update Logic The update logic is as follows.

```
if (hit) {
    l2_rrpv[set][way] = 0;
}
else {
    l2_rrpv[set][way] = maxRRPV - 1;  // Insert with RRPV 2
    if (isInstruction == 1)
        l2_rrpv[set][way] = 0;
}
```

Listing 3: Updating RRPV on cache hits and fills

When a cache line is accessed and results in a hit, its RRPV is set to 0, indicating that it is predicted to be re-referenced soon and should be protected from eviction.

When a line is missed and filled, its initial RRPV is set according to its type:

- **Instruction lines:** Inserted with RRPV 0, giving them the highest protection against eviction.
- **Data lines:** Inserted with RRPV 2, making them more susceptible to eviction than instruction lines.

Thus, the policy prioritizes the retention of instruction lines through both their initial RRPV assignment and the instruction-aware victim selection mechanism.

Table 7: IRRIP Performance and Speedup over LRU

Trace	IPC	Speedup	L2C Load MPKI	L2 Instruction MPKI	L2 Data MPKI
Trace 1	0.232182	0.9993×	17.70940	9.95880	7.75056
Trace 2	0.255837	1.0186×	23.29300	14.15580	9.13716
Trace 3	0.381997	0.9998×	3.92756	1.04076	2.88680

2.2 Task 2b

We use a set dueling inspired approach to dynamically choose between RRIP and IRRIP based on the instruction activity observed for each CPU. In our implementation, every 32nd L2 cache set is selected as a leader set. This gives us a sample of cache activity across the sets without requiring every set to maintain separate statistics.

To make the decision, we maintain four counters per CPU: total accesses, total misses, instruction accesses, and instruction misses. These counters are updated whenever an access to a leader set occurs. Using these statistics, we calculate the following metric:

$$R_{\text{instr}} = \frac{A_{\text{instr}} + M_{\text{instr}}}{A_{\text{total}} + M_{\text{total}}}$$

Here, A_{instr} and M_{instr} denote instruction accesses and misses, while A_{total} and M_{total} denote total accesses and misses. We set the threshold to $T = 0.2$. If the metric exceeds this threshold, the remaining sets use IRRIP; otherwise, they use RRIP. The idea is to give instruction lines higher priority when instruction activity is relatively high, helping protect them from eviction. When instruction activity is low, we retain RRIP instead.

We also keep the additional storage overhead small. Conventional RRIP uses only the l2_rrpv array to maintain the replacement state of cache lines. Our implementation retains this array and adds just four 64-bit counters per CPU to track the statistics needed for policy selection. This amounts to 32 bytes per CPU, or $32N$ bytes for N CPUs.

Table 8: IPC across Dynamic RRIP thresholds

Threshold	Trace 1	Trace 2	Trace 3
0.1	0.232203	0.255631	0.381997
0.2	0.232182	0.255669	0.382002
0.3	0.232264	0.255601	0.382017
0.4	0.231986	0.253367	0.381616

Table 9: L2C Load MPKI across Dynamic RRIP thresholds

Threshold	Trace 1	Trace 2	Trace 3
0.1	17.69030	23.37930	3.93080
0.2	17.69600	23.37700	3.93172
0.3	17.71280	23.37810	3.93196
0.4	17.58690	25.63400	4.04704

Table 10: L2 Instruction MPKI across Dynamic RRIP thresholds

Threshold	Trace 1	Trace 2	Trace 3
0.1	9.97064	14.29190	1.04532
0.2	9.97436	14.29120	1.04668
0.3	9.98992	14.29110	1.04656
0.4	10.52160	16.83400	1.20160

Table 11: L2 Data MPKI across Dynamic RRIP thresholds

Threshold	Trace 1	Trace 2	Trace 3
0.1	7.71964	9.08736	2.88548
0.2	7.72160	9.08576	2.88504
0.3	7.72292	9.08704	2.88540
0.4	7.06532	8.80004	2.84544

Table 12: Dynamic RRIP Results (Threshold = 0.2)

Trace	**IPC**	**L2C Load MPKI**	**L2 Instruction MPKI**	**L2 Data MPKI**
Trace 1	0.232182	17.69600	9.97436	7.72160
Trace 2	0.255669	23.37700	14.29120	9.08576
Trace 3	0.382002	3.93172	1.04668	2.88504

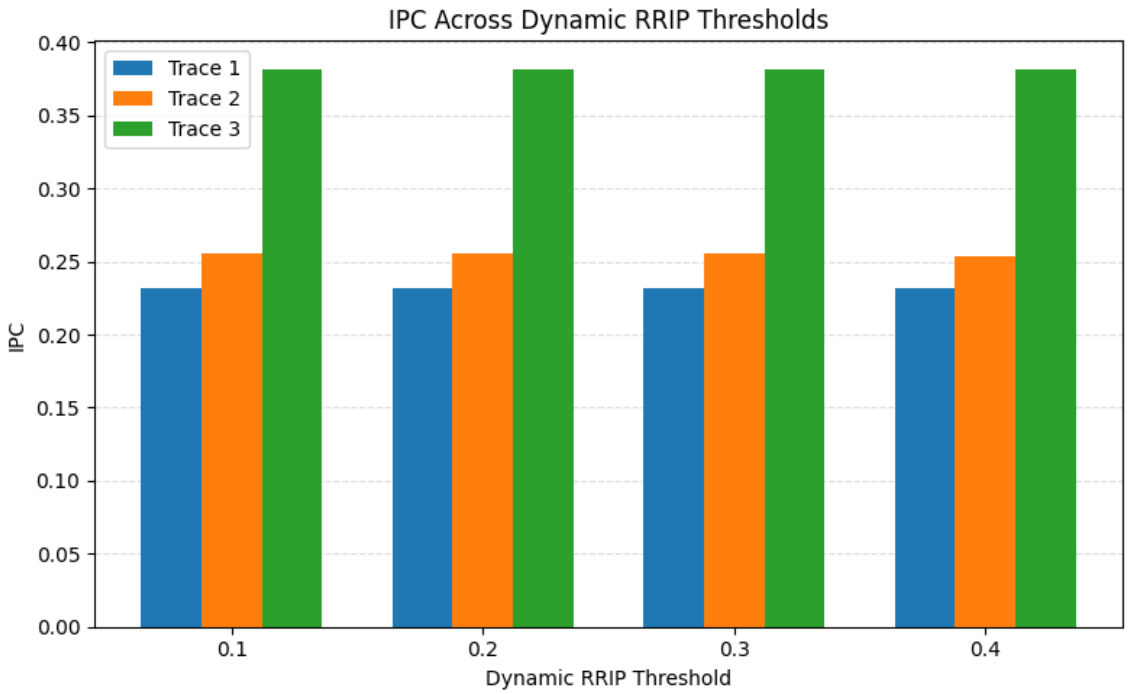


Figure 5: IPC across the traces across different threesholds

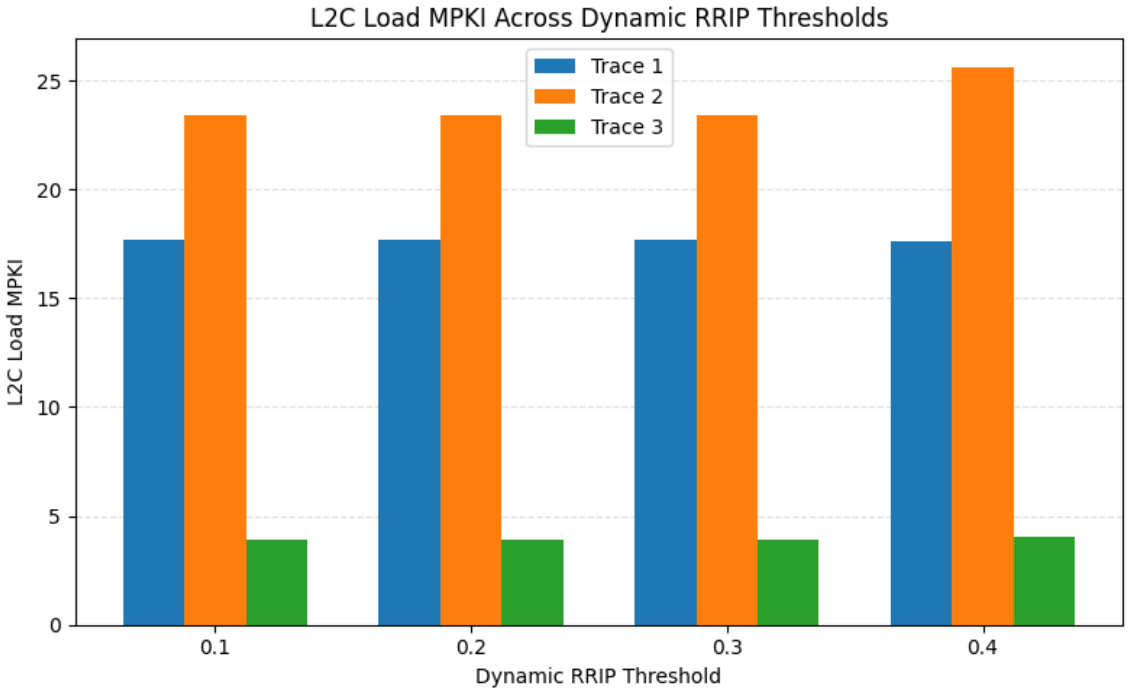


Figure 6: Load MPKI across the traces across different threesholds

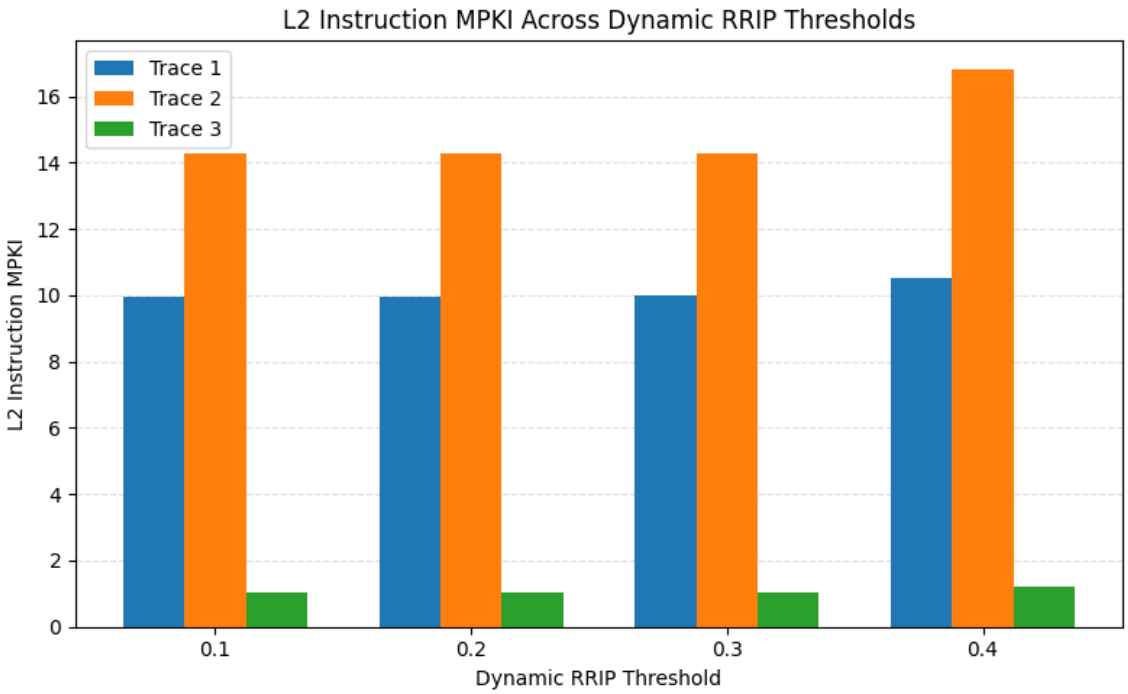


Figure 7: L2C Instruction MPKI across the traces across different threesholds

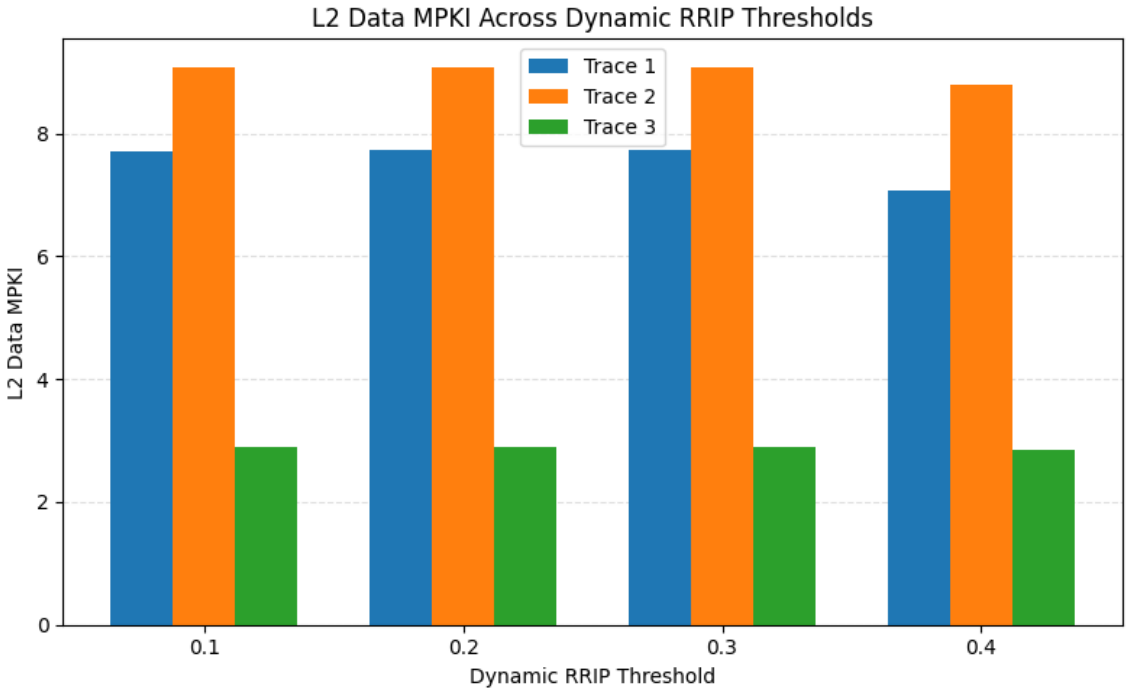


Figure 8: L2C data MPKI across the traces across different thresholds

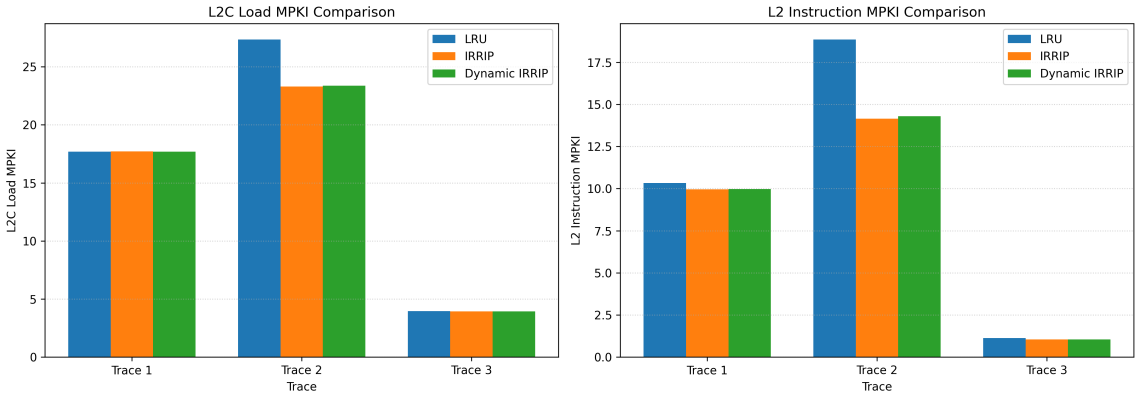


Figure 9: L2C total mpki (threshold=0.2)

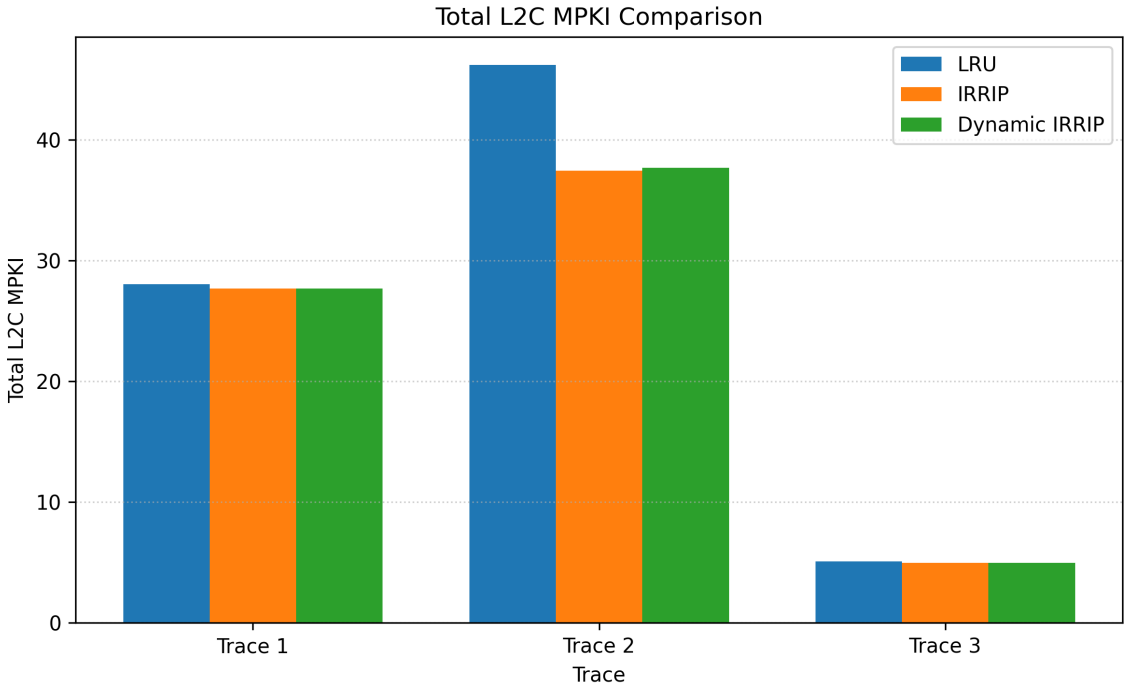


Figure 10: L2C mpki (threshold=0.2)

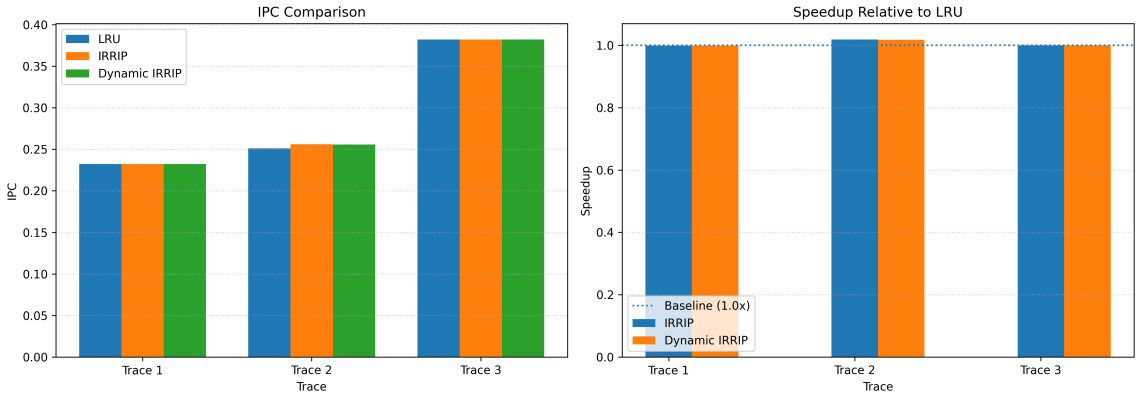


Figure 11: ipc and threshold (threshold=0.2)

3 Task 3

We insert prefetched lines with an RRPV of 2 and assign them the maximum RRPV of 3 if they are accessed. During victim selection under IRRIP, prefetched, non-instruction lines with the maximum RRPV are given the highest priority for eviction, followed by other non-instruction lines and then instruction lines. This ensures that prefetched lines are not protected for too long, even if they are used, helping prevent cache pollution while preserving valuable instructions and data. For all other types of accesses, we follow the normal dynamic I-RRIP policy without any changes.

Table 13: Baseline Performance (No Prefetcher)

Trace	IPC	L1D MPKI	L2C Load MPKI	L2 Inst. MPKI	L2 Data MPKI
Trace 1	0.232345	15.1755	17.69560	10.33770	7.35796
Trace 2	0.251158	18.7336	27.35680	18.85330	8.50352
Trace 3	0.382065	5.03724	3.95424	1.12408	2.83016

Table 14: SPP Prefetcher with Dynamic RRIP Performance

Trace	IPC	Speedup	Accuracy (%)	Coverage (%)	L1D MPKI
Trace 1	0.233326	1.0042×	64.1238	-1.7924	15.4476
Trace 2	0.252105	1.0038×	62.2749	-0.3936	18.8073
Trace 3	0.383175	1.0029×	46.7996	-2.7879	5.1776

Table 15: SPP Prefetcher with Dynamic RRIP Performance

Trace	L2C Load MPKI	L2 Inst. MPKI	L2 Data MPKI
Trace 1	17.14990	10.53770	6.61216
Trace 2	25.64240	17.71990	7.92244
Trace 3	3.75372	1.19780	2.55592

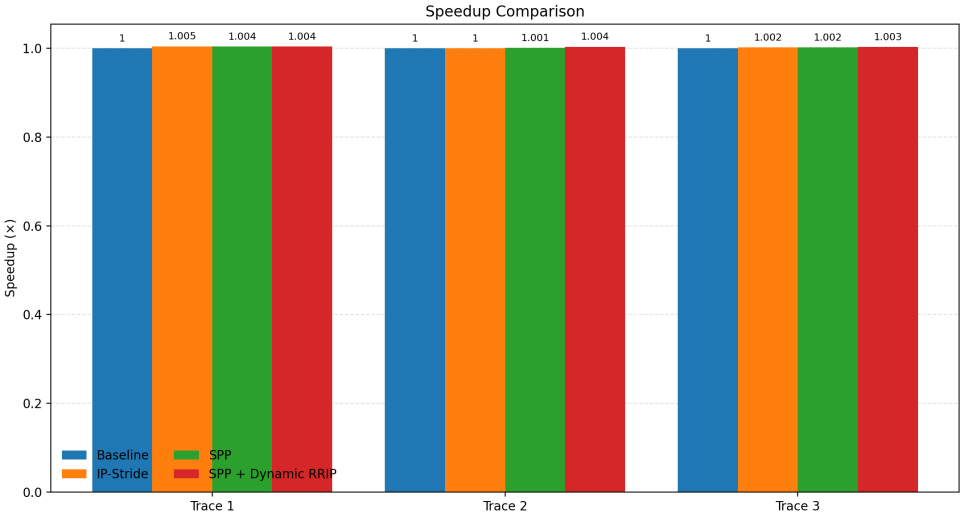


Figure 12: Speedup comparison of the baseline, IP-Stride, SPP, and SPP with Dynamic RRIP across the three traces.

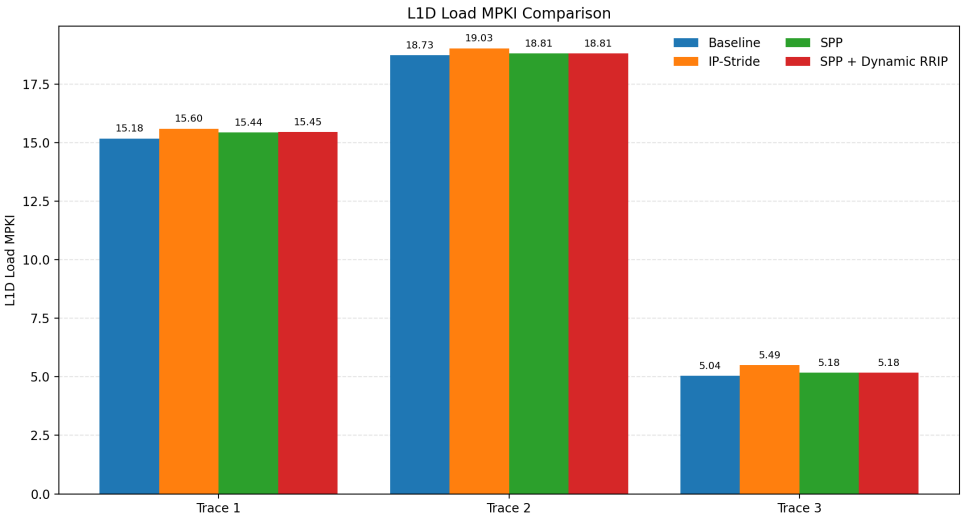


Figure 13: Comparison of L1D load MPKI for the baseline, IP-Stride, SPP, and SPP with Dynamic RRIP.

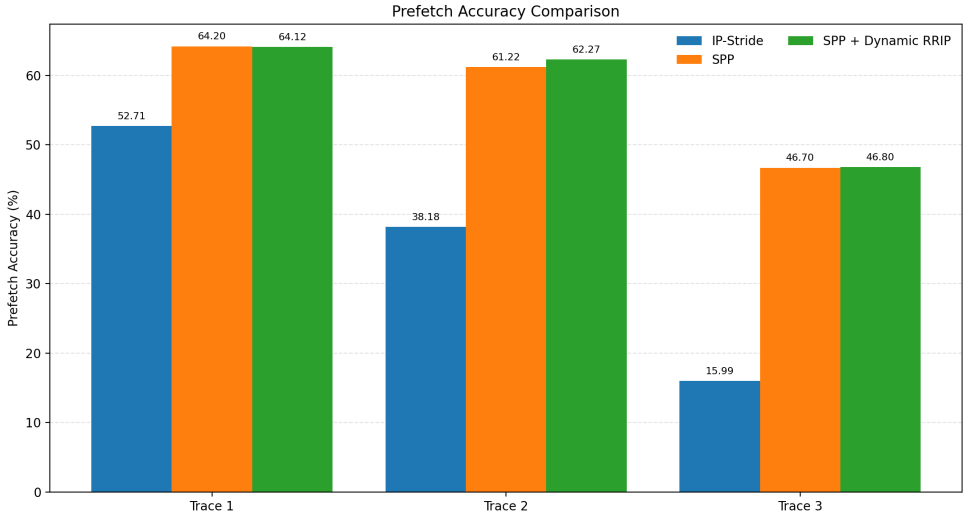


Figure 14: Prefetch accuracy comparison of IP-Stride, SPP, and SPP with Dynamic RRIP across the three traces.

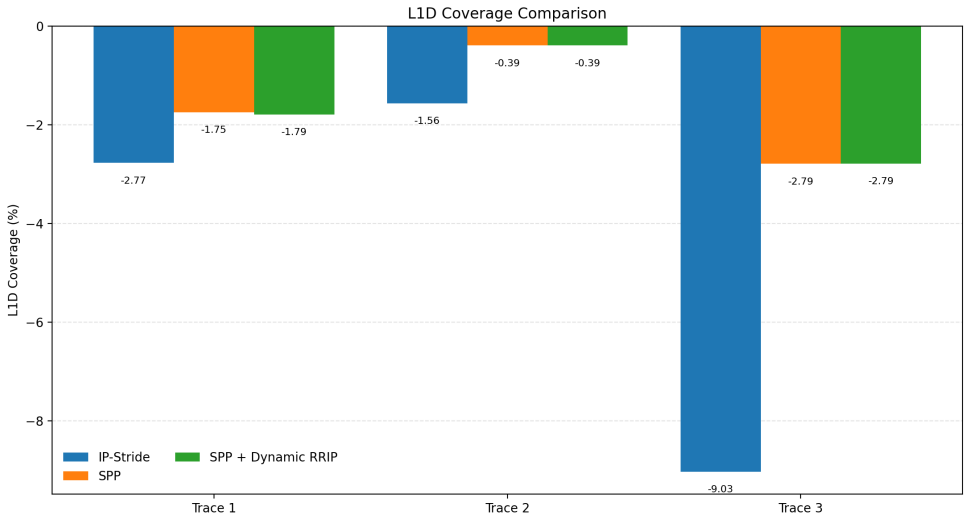


Figure 15: L1D coverage comparison of IP-Stride, SPP, and SPP with Dynamic RRIP across the three traces.

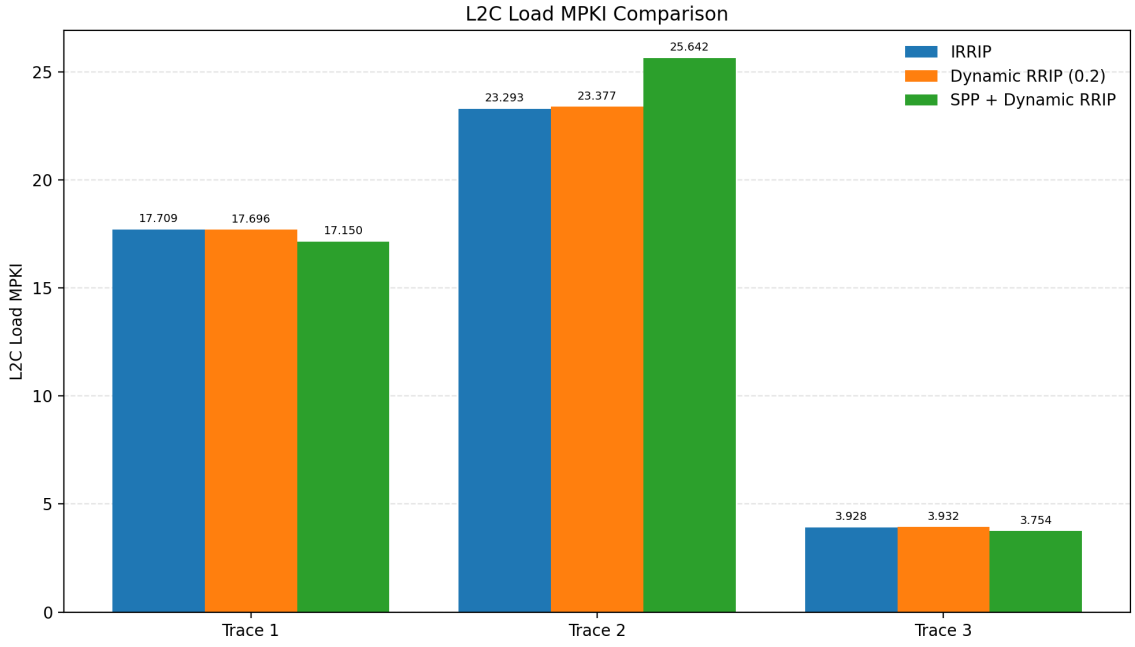


Figure 16: L2 data mpki comparison

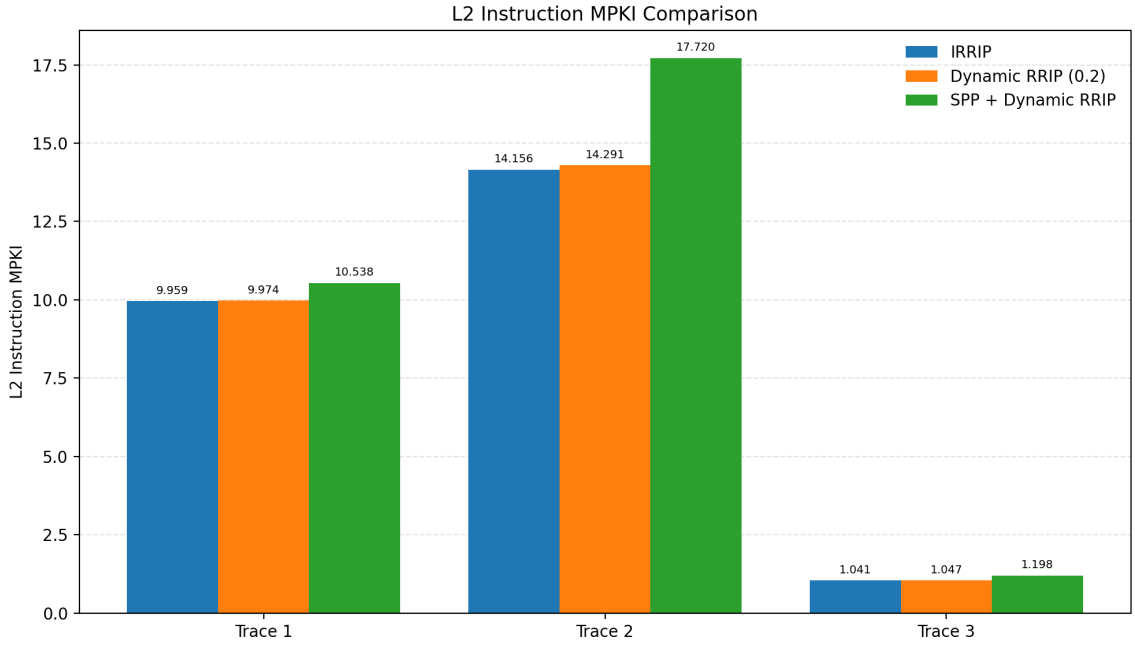


Figure 17: L2 instruction mpki comparison

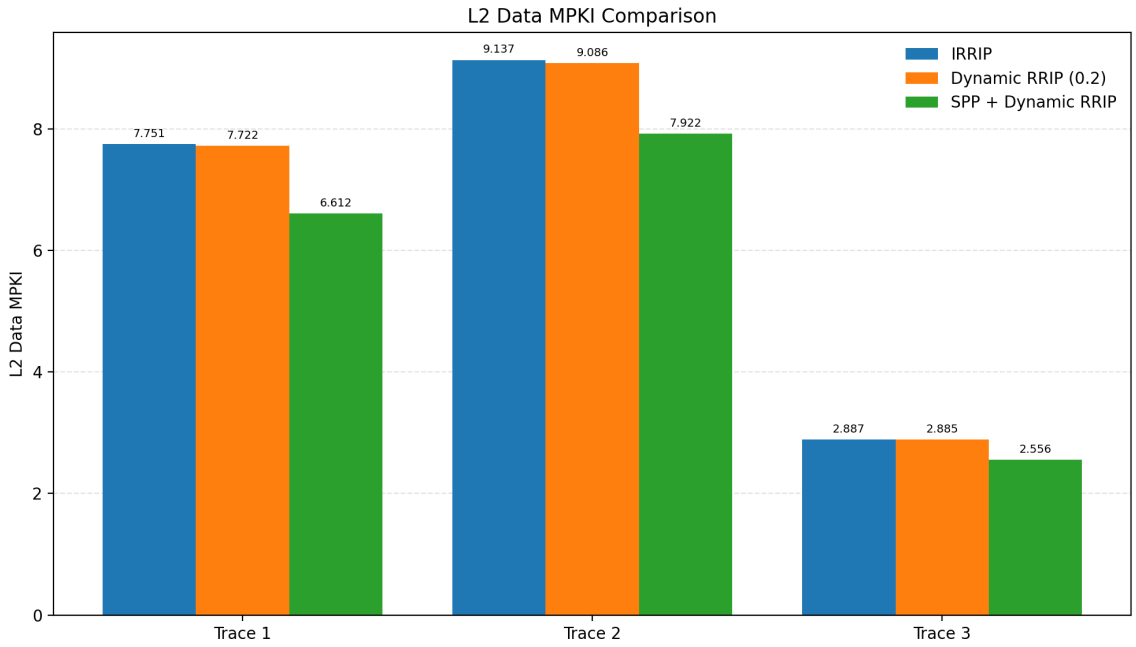


Figure 18: L2 data mpki comparison